Ex No: 1

Date: 17/2/26

BASIC NETWORKING COMMANDS IN LINUX AND WINDOWS OPERATING SYSTEMS

AIM:

To study and understand the basic networking commands used in Linux and windows operating systems

COMMANDS OBSERVED:

NO	Purpose	Linux Commands	Windows Commands
1	Display IP configuration	ipconfig or ip addr	ip config
2	check Network Connectivity	ping <host>	Ping <host>
3	trace Route	traceroute <host>	tracert <host>
4	Display Routing Table	route -n or ip route	route print
5	Display Network Connections	net stat -a	net stat -a
6	DNS LookUp	nslookup <domain>	nslookup <domain>
7	Display ARP cache	arp -a	arp -a
8	Display hostname	hostname	hostname
9	Secure file transfer over SSH	sftp	sftp
10	Dig DNS lookup	dig <domain>	nslookup <domain>

NO	Purpose	Linux commands	Windows command
11	DNS Information	dig	nslookup
12	Host lookup	host	nslookup
13	Socket statistics	ss	netstat -a no
14	Download Web Content	Curl	Curl
15	Download files	Wget	curl -O
16	Packet Capture	tcpdump	Pktmon
17	Network diagnostic	mtr	Path ping
18	Network Management	nmchi	netsn
19	Network adapter info	ethtool	getmac
20	Wireless Configuration	iwconfig	netsh wlan show interfaces

RESULT:

The Basic networking commands in linux and windows were successfully studied and observed

EXP. NO: 2

DATE: 17/7/26

LEARNING AND ASSIGNMENT OF IP ADDRESS MANUALLY TO COMPUTER

AIM:

To learn the concept of IP addressing and assign a manual IP address to a computer for network communication

PROCEDURE:

1. Turn on the Computer and open Control Panel
2. Go to Network and Sharing Center and Select change adapter Settings.
3. Right-click the active network adapter and Select Properties
4. Select Internet Protocol Version 4 (TCP/IPV4) and Click Properties
5. Choose Use the following IP address
6. Enter the required network details:
   - IP Address: 192.168.1.10
   - Subnet Mask: 255.255.255.0
   - Default Gateway: 192.168.1.1
7. Enter the Preferred DNS Server (eg 8.8.8.8)
8. Click OK to Save the Settings.
9. Open the Command Prompt and type ipconfig to verify the assigned IP address

RESULT:

Therefore the Computer was successfully configured with a manual IP address and network connectivity was verified

OUTPUT:

```
C:\> ipconfig

IPv4 Address . . . . . : 192.168.1.10
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : 192.168.1.1
DNS Servers . . . . . : 8.8.8.8
```

EXP. NO: 3

DATE: 23/7/24

STUDY OF DIFFERENT TYPES OF NETWORK CABLES AND CRIMPING OF CABLE WITH RJ45 CONNECTOR

AIM:

To Study different types of network cables and perform crimping of a UTP cable using an RJ45 Connector

MATERIALS REQUIRED:

- UTP (cat 5e / cat6) cable
- RJ45 Connector
- Crimping tool
- Wire stripper
- Cable cutter
- LAN cable tester

TYPES OF NETWORK CABLES:

=> UTP (unshielded Twisted Pair): Most Commonly used for Ethernet Networks.

Speed : upto 40Gbps
Range : 100meters

=> STP (shielded Twisted Pair): Has shielding to reduce electromagnetic interference

Speed : upto 40Gbps
range : 100meters

=> coaxial cable : used for cable Tv and older Networking systems.

Speed : upto 10Gbps
range : 200 to 500 meters

TYPES OF NETWORK CABLES

Choose the Right Cable for Reliable & High-Performance Networks

1 UNSHIELDED TWISTED PAIR (UTP) CABLES



Most common cable for LAN, Ethernet, and office networks.

STRUCTURE

- Outer Jacket
- Twisted Pairs

KEY BENEFITS

- Commonly used in homes and offices
- Flexible, affordable, and easy to use
- For Ethernet and VoIP applications

2 SHIELDED TWISTED PAIR (STP) CABLES



Ideal for environments with high EMI and interference.

STRUCTURE

- Outer Jacket
- Foil Shielding
- Twisted Pairs

KEY BENEFITS

- Protect against interference and crosstalk
- Require proper grounding
- Ideal for environments with heavy EMI such as factories and industrial setups

3 COAXIAL CABLES



Used for traditional broadband, CCTV, and RF transmissions.

STRUCTURE

- Outer Protective Jacket
- Metallic Shield
- Copper Conductor

KEY BENEFITS

- Highly resistant to signal obstruction
- Great for long-distance transmissions
- Commonly used in data centers, ISP networks, and radio/broadcast applications

4 FIBER OPTIC CABLES



Built for speed and performance in modern networks.

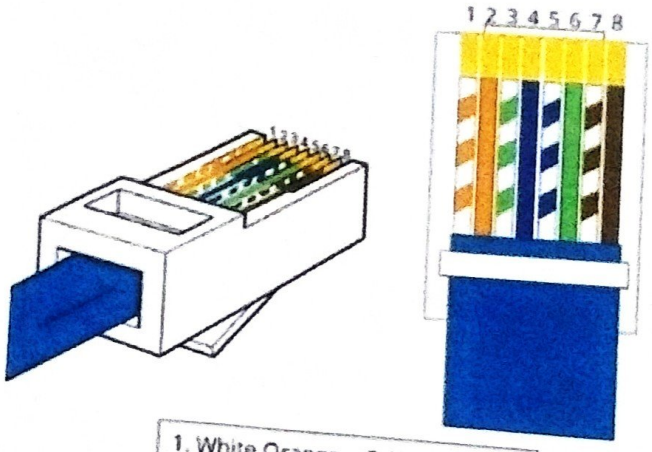
STRUCTURE

- Outer Jacket
- Glass Fiber Strands
- Light Signal Transmission

KEY BENEFITS

- Immune to electromagnetic interference
- Ideal for ultra-fast, long-range data transmission
- Widely used in data centers, ISP networks, and high-bandwidth applications

Tip: Selecting the right cable improves network performance, reduces downtime, and ensures a reliable network infrastructure.



1. White Orange
2. Orange
3. White Green
4. Blue
5. White Blue
6. Green
7. White Brown
8. Brown

RJ45 CONNECTOR

=> Fiber optic cable: uses light to transmit data at very high speeds over long distances.

Speed: 100+ Gbps Range: upto 40 Kilometers

CRIMPING PROCEDURE:

* Cut the cable to the required length
* Strip about 2-3cm of the outer insulation
* Untwist and arrange the wires according to the T568B standard:
  - White/orange
  - orange
  - White/Green
  - Blue
  - White/Blue
  - Green
  - White/Brown
  - Brown
* Trim the wires evenly
* Insert the wires fully into the RJ45 connector
* Use the crimping tool to crimp the connector
* Test the cable using a LAN cable tester.

RESULT:

Thus the network cable was successfully crimped with an RJ45 connector and tested for proper connectivity

EX.NO: 4

DATE: 30/7/26

IMPLEMENT PACKET SNIFFING USING RAW SOCKETS IN PYTHON

AIM:

To Implement a Packet Sniffer using raw sockets in Python.

REQUIREMENTS

- Python 3.x
- Raw Socket Support
- Windows/Linux operating System (Run with administrator or root privileges).

PACKET SNIFFING:

Packet Sniffing is the process of Capturing and analyzing network packets as they travel across a network. In python, raw sockets provide direct access to network packets, allowing programmers to inspect packet headers and payloads. This technique is widely used for network monitoring, troubleshooting and Security analysis.

OUTPUT SAMPLE 1

Packet Captured
Source IP : 192.168.1.10
Destination IP : 192.168.1.1
Protocol : TCP
Packet Length : 60 bytes

OUTPUT SAMPLE 2

Packet Captured
Source IP : 8.8.8.8
Destination IP : 192.168.1.10
Protocol : ICMP
Packet Length : 84 bytes

PROCEDURE:

1. Import the required Python modules
2. Create a raw Socket
3. Bind the socket to the network interface
4. Continuosly listen for incoming packets
5. Capture the packet data
6. Display packet information such as Source, destination and protocol
7. Repeat until the program is terminated

PROGRAM

```
import socket
s = socket.socket(socket.AF_PACKET, socket.SOCK_RAW,
                  socket.ntohs(3))
while True:
    pkt = s.recvfrom(65535)[0]
    if len(pkt) > 34 and pkt[12:14] == b'\x08\x00':
        print(f"Packet Captured \n Source IP : {socket.inet_ntoa(pkt[26:30])}
        \n Destination IP : {socket.inet_ntoa(pkt[30:34])} \n Protocol :
        {pkt[23]} \n Packet Length : {len(pkt)} \n")
```

RESULT:

Thus the packet Sniffer using raw socket in python implemented Successfully.

Command: ping google.com

OUTPUT:

Pinging google.com [142.250.77.110] with 32 bytes of data:
Reply from 142.250.77.110: bytes=32 time=13ms TTL=119
Reply from 142.250.77.110: bytes=32 time=4ms TTL=119
Reply from 142.250.77.110: bytes=32 time=7ms TTL=119

Ping statistics for 142.250.77.110:

Packets: sent=4, Received=4, Lost=0 (0% loss),
Approximate round trip times in milli-seconds:
Minimum=4ms, Maximum=13ms, Average=7ms

Command: ping -n 3 google.com

OUTPUT:

Pinging google.com [142.250.77.110] with 32 bytes of data:
Reply from 142.250.77.110: bytes=32 time=20ms TTL=119
Reply from 142.250.77.110: bytes=32 time=25ms TTL=119
Reply from 142.250.77.110: bytes=32 time=25ms TTL=119

Ping statistics for 142.250.77.110:

Packets: Sent=3, Received=3, Lost=0 (0% loss),
Approximate round trip times in milli-seconds:
Minimum=20ms, Maximum=25ms, Average=23ms

EXP.NO: 5

DATE: 24/7/26

DEVELOP A CUSTOMIZED PING COMMAND TO TEST SERVER CONNECTIVITY

AIM:

To develop a customized Ping commands to test Server Connectivity

PURPOSE OF USING PING COMMAND:

Ping Commands are used to

- Verify whether a Server or host is reachable
- Test the connectivity between two devices
- Measure the Round Trip Time (RTT) or latency.
- Check for packet loss during communication

PURPOSE OF PING COMMAND:

-> Ensure that communication with the destination Server is Successful

-> Evaluate the quality and reliability of a network connection

-> Detects delays in data transmission

-> Identify packet loss that may affect network performance.

Command: ping -l 1000 google.com

OUTPUT:

Pinging google.com [142.250.67.46] with 1000 bytes of data:
Request timed out.
Reply From 142.250.67.46 : bytes = 1000 time = 3ms TTL = 118
Reply from 142.250.67.46 : bytes = 1000 time = 8ms TTL = 118
Reply from 142.250.67.46 : bytes = 1000 time = 5ms TTL = 118

Ping statistics for 142.250.67.46:

Packets: Sent = 4, Received = 3, Lost = 1 (25% Loss),
Approximate round trip times in milli-seconds:

Minimum = 3ms, Maximum = 8ms, Average = 5ms

Command: ping -w 3000 google.com

OUTPUT:

Pinging google.com [142.250.67.46] with 32 bytes of data:
Reply from 142.250.67.46 : bytes = 32 time = 3ms TTL = 118
Reply from 142.250.67.46 : bytes = 32 time = 5ms TTL = 118
Reply from 142.250.67.46 : bytes = 32 time = 9ms TTL = 118
Reply from 142.250.67.46 : bytes = 32. time = 12ms TTL = 118

Ping statistics for 142.250.67.46:

Packets: Sent = 4, Received = 4, Lost = 0 (0% Loss)
Approximate round trip times in milli seconds

Minimum = 3ms, Maximum = 12ms, Average = 7ms

CONNECTION ESTABLISHMENT PROCEDURE

1. Open the command prompt or terminal
2. Execute the required customized ping command
3. Observe the transmitted and received packet information
4. Record the response time and packet loss
5. Analyze the network performance based on the obtained results.

CUSTOMIZED PING COMMANDS.

Command	Purpose
ping google.com	check connectivity with the server
ping -n 10 google.com	Sends 10 ICMP Echo Requests.
Ping -t google.com	continuously sends ping request until stopped
ping -l 1000 google.com	send packets with a payload size of 1000 bytes
ping -w 3000 google.com	waits upto 3000ms for each reply

RESULT:

Thus customized Ping Commands are developed and the server connectivity is tested successfully

Ex.NO: 9

DATE: 13/8/26

IMPLEMENTATION OF SUBNETTING IN CISCO PACKET TRACER

AIM:

To implement the Subnetting and Configure a Subnetted network using CISCO Packet Tracer.

THEORY:

Subnetting divides a large network into smaller networks called Subnets. It improves IP address utilization network management, security and reduces network traffic.

For the network ~~192.168~~ 192.168.1.0/24, a /27 Subnet mask is used.

Subnet mask: 255.255.255.224

Number of Subnets: 8

Usable hosts per Subnet: 30

ALGORITHM:

1. Open CISCO Packet Tracer and Create the required network topology
2. Connect routers, Switches and PCs using appropriate cables
3. Divide the network 192.168.1.0/24 into Subnets using /27
4. Configure the router interfaces with the assigned IP addresses.
5. Configure the Switch ports in access mode

Ex.NO: 9

DATE: 13/8/26

Aim:

To imple
Subnetted

THEORY:

Subnetting
networks ca
network ma

For the
id used.

Subnet mas
Number of
Usable h

ALGORITH

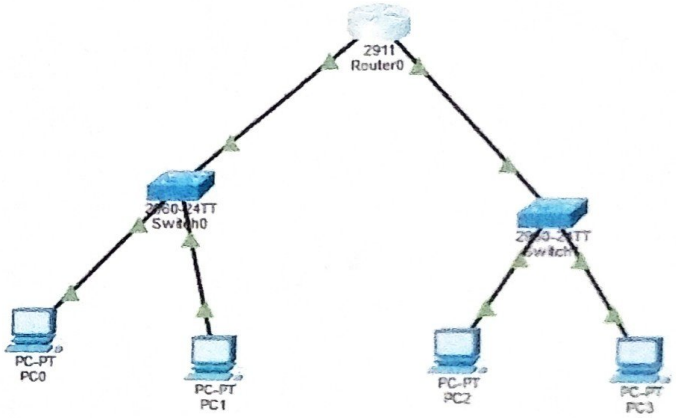
1. Open
Network

2. Conn
cables

3. Di
wi

4. Con
IP

5.




```
Router>enable
Router#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     192.168.1.1     YES manual up                    up
GigabitEthernet0/1     192.168.2.1     YES manual up                    up
GigabitEthernet0/2     unassigned      YES unset  administratively down down
Vlan1                  unassigned      YES unset  administratively down down
```

```
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 192.168.1.10

Pinging 192.168.1.10 with 32 bytes of data:

Reply from 192.168.1.10: bytes=32 time=3ms TTL=128
Reply from 192.168.1.10: bytes=32 time<1ms TTL=128
Reply from 192.168.1.10: bytes=32 time=6ms TTL=128
Reply from 192.168.1.10: bytes=32 time=4ms TTL=128

Ping statistics for 192.168.1.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 6ms, Average = 3ms

C:\>
```

INPUT:

Network address : 192.168.1.0/24

Number of Subnets : 8

Usable hosts per Subnet : 30

Subnet mask: 255.255.255.224 (/27)

Devices : 2 Routers, 2 switches and PCs

Ping 192.168.1.1

OUTPUT:

Reply from 192.168.1.1: bytes = 32 time <1ms
TTL = 255

Reply from 192.168.1.1 : bytes = 32 time <1ms
TTL = 255

Reply from 192.168.1.1 : bytes = 32 time <1ms
TTL = 255

Reply from 192.168.1.1 : bytes = 32 time <1ms
TTL = 255

6. Assign IP address, Subnet mask and default gateway to each PC.

7. Configure routing between the routers if required.

8. Test the Connectivity using the ping command

IP ADDRESS ASSIGNMENT

Device	Interface	IP Address
Router1	G0/0	192.168.1.1
Router2	G0/1	192.168.2.1
Router2	F0/0	192.168.3.1
Router2	F0/1	192.168.4.1
PC1	—	192.168.1.11
PC2	—	192.168.1.12
PC3	—	192.168.1.13
PC4	—	192.168.1.14
PC5	—	192.168.1.15

COMMANDS FOR ROUTERS

ROUTER 1

enable
configure terminal

interface gigabitethernet 0/0
ip address 192.168.1.1 255.255.255.224
no shutdown
exit

interface gigabitethernet 0/1
ip address 192.168.2.1 255.255.255.224
no shutdown
exit

ROUTER 2

enable
configure terminal

interface fastethernet 0/0
ip address 192.168.3.1 255.255.255.224
no shutdown
exit

interface fastethernet 0/1
ip address 192.168.3.1 255.255.255.224
no shutdown
exit

SWITCH CONFIGURATION

enable
configure terminal
interface fastethernet 0/11
switchport mode access
exit
interface fastethernet 0/12
switchport mode access
exit

PC CONFIGURATION

For each PC configure:

IP Address: 192.168.x.11
Subnet Mask: 255.255.255.224
Default Gateway: 192.168.x.1
Assign different host addresses (.11, .12, .13, .14, .15)
to the PCs according to their Subnet

RESULT:

Therefore a Simple topology with one configured router two Switches and PCs using Cisco Packet tracer is done.

Ex. NO: 10

DATE: 19/8/26

DESIGN A CUSTOMIZE SWITCH WITH NETWORK MODULES USING CISCO PACKET TRACER

AIM:

To design and configure internetworks using routers, wireless router, DHCP Server and internet cloud in Cisco packet Tracer and Verify Connectivity

PROCEDURE:

1. Open Cisco Packet Tracer and create the required network topologies.
2. Configure Router1 with
   - Fa 0/0 -> 192.168.10.1 /24
   - Fa 0/1 -> 192.168.10.1 /24
   - Enable Interfaces using no shutdown.
3. Configure
   - PC0 -> 192.168.10.2, Gateway 192.168.10.1
   - PC1 -> 192.168.20.2, Gateway 192.168.20.1
4. Connect PCs to the router using Copper Straight-through Cables and Send a Simple PDU from PC0 to PC1
5. Create the Second Topology using PC, wireless Router, Laptop, Cable Modem, internet cloud and Cisco.com Server.
6. Connect the devices using Straight-through And Coaxial Cables as required.

7. Configure Wireless Router:

- LAN IP: 192.168.0.1
- Mask: 255.255.255.0
- SSID: HomeNetwork
- Enable DHCP
- DNS: 208.67.220.220

8. Connect the laptop to HomeNetwork using the WPC300N wireless module

9. Configure PC and Laptop to obtain IP addresses through DHCP

10. Configure Internet cloud and Cisco.com Server with the required modules DHCP and DNS Services.

11. Configure Cisco.com Server:

- IP: 208.67.220.220
- Mask: 255.255.255.0
- Gateway: 208.67.220.1

12. Verify Connectivity Using:

ipconfig /renew

ping Cisco.com

IP CONFIGURATION

DEVICE	IP ADDRESS	SUBNET MASK	GATEWAY
Router Fa0/0	192.168.10.1	255.255.255.0	—
Router Fa0/1	192.168.20.1	255.255.255.0	—
PC0	192.168.10.2	255.255.255.0	192.168.10.1
PC1	192.168.20.2	255.255.255.0	192.168.20.1
Wireless Router LAN	192.168.0.1	255.255.255.0	—
PC	DHCP (192.168.0.x)	255.255.255.0	192.168.0.1
Laptop	DHCP	255.255.255.0	192.168.0.1
Cisco.com Server	208.67.220.220	255.255.255.0	208.67.220.1

WIRELESS ROUTER & DHCP:

Wireless Router provides WiFi, SSID and internet connectivity, DHCP automatically assigns IP, gateway and DNS.

Pinging Cisco.com [208.67.220.220]

Reply from 208.67.220.220 : bytes = 32 time < 1ms TTL = 255

Reply from 208.67.220.220 : bytes = 32 time < 1ms TTL = 255

Reply from 208.67.220.220 : bytes = 32 time < 1ms TTL = 255

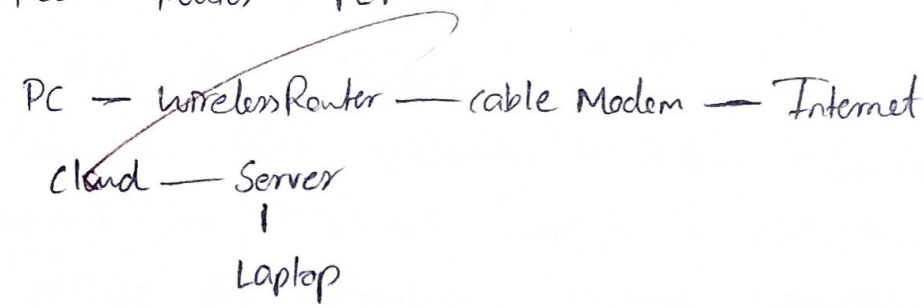
Reply from 208.67.220.220 : bytes = 32 time < 1ms TTL = 255

Packet Sent = 4 , Received = 4 , Lost = 0 (0% Loss)

SIGNIFICANCE OF DHCP:

DHCP provides network settings automatically and reduces manual configuration and IP conflicts

NETWORK TOPOLOGY:



RESULT:

The internetworks were successfully designed and configured in Cisco Packet Tracer. Connectivity was verified using PDU and ping and DHCP successfully assigned IP configuration

EX NO: 7

DATE: 20/8/26

DEVELOP A SIMPLE CALCULATOR USING XMLRPC

AIM:

To develop a Simple Calculator using XMLRPC (XML Remote Procedural Call)

INTRODUCTION TO XML-RPC:

XML-RPC (Remote Procedural Call) is a protocol that allows a client application to execute a function or procedure on a remote Server over a network. It uses XML to encode the request and response messages and commonly works over HTTP for communication. XML-RPC follows a Simple Client-Server architecture. Where the client sends a required containing the method name and parameters and the Server processes the request and sends back the result. It can be used to build distributed applications such as a remote calculator, where arithmetic operations are performed by the Server based on request from the client.

PROCEDURE:

1. Create an XML-RPC Server that provides methods for Basic addition, Subtraction, multiplication and division
2. Start the Server on a Specified host and port
3. Create an XML-RPC client to connect the Server.
4. Get the required numbers and operation from the User.

5. Send the request from the client to the Server.
6. The Server performs the Calculation and returns the result.
7. Display the result on the Client Side
8. Test the Calculator with different inputs.

SERVER SIDE PROGRAM:

```
from xmlrpc.server import SimpleXMLRPCServer

def add(x,y): return x+y
def subtract(x,y): return x-y
def multiply(x,y): return x*y
def divide(x,y): return x/y if y!=0 else "Error: Div by 0"

server = SimpleXMLRPCServer(("localhost", 8000))
print("XML-RPC Calculator Server is running on port 8000...")
server.register_function(add, "add")
server.register_function(subtract, "subtract")
server.register_function(multiply, "multiply")
server.register_function(divide, "divide")
server.serve_forever()
```

OUTPUT

```
xmlrpc Calculator client
  1. Add
  2. Subtract
  3. Multiply
  4. Divide
  5. Exit
Choose an operation (1-5)
1
Enter first number : 20
Enter second number : 30
Result : 50
2
Enter first number : 5
Enter Second number : 2
Result : 3
3
Enter first number : 5
Enter Second number : 4
Result : 20
4
Enter first number : 10
Enter second number : 2
Result : 5
5
Exiting
```

CLIENT SIDE PROGRAM:

```
import xmlrpc.client
server = xmlrpc.client.ServerProxy("http://localhost:8000")
while True:
 print("\n--XML-RPC Calculator Client--")
 print("1. Add")
 print("2. Subtract")
 print("3. Multiply")
 print("4. Divide")
 print("5. Exit")
 choice = input("choose an operation(1-5): ")
 if choice == '5':
    print("Exiting Calculator. Goodbye")
    break
 if choice not in ['1','2','3','4']:
    print("Invalid choice! Please Select 1,2,3,4 or 5") continue
 try:
  num1 = float(input("Enter first number: "))
  num2 = float(input("Enter second number: "))
 except ValueError:
    Print("Error: Please enter valid numbers") continue
 if choice == '1':
    print("Result:", server.add(num1,num2))
 elif choice == '2':
    print("Result", server.subtract(num1,num2))
 elif choice == '3':
    print("Result:", server.multiply(num1,num2))
 elif choice == '4':
    print("Result:", server.divide(num1,num2))
```

RESULT:

Thus the Simple Calculator is implemented using XML RPC

Lab-No:6

21/9/26

ERROR CORRECTION AT DATA LINK LAYER

Aim:

Write a Program to implement error detection and correction using HAMMING code concept Make a test run to input data stream and verify error correction feature

SENDER PROGRAM WITH BELOW FEATURES.

1. Input to sender file should be a text of any length. Program should convert the text to binary.
2. Apply Hamming code concept on the Binary data and add redundant bits to it.
3. Save this output in a file called channel.

RECEIVER PROGRAM WITH BELOW FEATURES:

1. Receiver program should read the input from the Channel file
2. Apply hamming code on the binary data to check for Errors
3. If there is an error, display the position of the error.
4. Else, remove the redundant bits and convert the binary data to ASCII data and display the output.

SENDER PROGRAM (Sender.py)

```
def encode(d):
    c = [0] * 13
    j = 0
    for i in range(1, 13):
        if i not in (1, 2, 4, 8):
            c[i] = d[j]
            j += 1
    for p in (1, 2, 4, 8):
        for i in range(1, 13):
            if i & p and i != p:
                c[p] ^= c[i]
    return ''.join(map(str, c[1:]))

s = input("Enter message: ")
b = ''.join(format(ord(x), '08b') for x in s)
with open("channel", "w") as f:
    for i in range(0, len(b), 8):
        f.write(encode(list(map(int, b[i:i+8]))) + '\n')
print("Data encoded and saved.")
```

RECEIVER PROGRAM

```
def decode(s):
    c = [0] + list(map(int, s))
    e = 0
    for p in (1, 2, 4, 8):
        if sum(c[i] for i in range(1, 13)
if (i % p) % 2:
            e += p
    if e:
        print("Error at position", e)
        c[0] ^= 1
        return ''.join(str(c[i]) for i in range(1, 13)
                       if i not in (1, 2, 4, 8))
with open("channel") as f:
    b = ''.join(decode(s.strip()) for s in f)
print("Received message:", ''.join(chr(int(b[i:i+8], 2))
                                   for i in range(0, len(b), 8)))
```

Input:

Sender Program

Enter message: Hi

Data encoded and saved

OUTPUT:

Received message: Hi

RESULT:

Thus the Error Correction at Data Link layer is executed

EXP.NO:8

24/9/26

NMAP HOST DECLIVERY

Aim:

To discover live hosts using Nmap Scans (ARP, ICMP, TCP/UDP) on the Try HackMe platform.

PROCEDURE:

1. Open the Try HackMe Nmap from Nmap room: Nmap Rooms
2. Start the AttackBox provided by the platform
3. Identify the target IP address or network range given in the authorized lab.
4. Perform host discovery using Nmap.
5. Use ARP, ICMP, TCP and UDP Scanning methods to identify active host.
6. Observe the Scan results and record the live hosts discovered.

COMMANDS:

nmap -sn 10.10.210.0/24

ARP Scan:

nmap -PR -sn 10.10.210.0/24

ICMP Scan:

nmap -PE -sn 10.10.210.0/24

TCP SYN PING:

nmap -PS80,443 -sn <target>

UDP PING:

nmap -PU53,161 -sn <target>

OUTPUT:

Nmap Scan report for 10.10.210.6
Host is up

Nmap scan report for 10.10.210.10
Host is up..

Nmap done : 256 IP addresses Scanned

INPUT:

Target Network: 10.10.210.0/24

Scan Type: ARP host discovery

RESULT:

Thus Live host discovery using Nmap with ARP, ICMP, TCP and UDP Scanning techniques was studied Sucessfully.

EXP NO: 11

24/9/26

ROUTING AT NETWORK LAYER

a) Simulate Static Routing Protocol Configuration using CISCO Packet Tracer.

Aim: To simulate static Routing Protocol Configuration using Cisco Packet Tracer.

PROCEDURE:

1. Open Cisco Packet Tracer
2. Place two routers two switches and two PCs
3. Connect static routes on devices using suitable cables
4. Assign IP addresses to the PCs and router interface
5. Configure static route on both routers.
6. Enable the router interface using no shutdown
7. Test connectivity between the PCs using the ping command.

STATIC ROUTING:

Static Routing is a networking method where a network administrator manually creates and configures fixed paths in a router's routing table

IP configuration

Device	IP address	Subnet Mask
PC0	192.168.1.2	256.255.255.0
Router 1 LAN	192.168.1.1	255.255255.0
Router 2 LAN	192.168.2.1	255.255.255.0
Router 1 Link	10.0.0.1	255.255.255.252
Router 2 Link	10.0.0.2	255.255.255.252
PC1	192.168.2.2	255.255.255.0

CONFIGURATION COMMANDS

Static Routing

Router 1

```
enable
configure terminal
ip route 192.168.2.0 255.255.255.0
10.0.0.2
end
```

Router 2

```
enable
configure terminal
ip route 192.168.1.0 255.255.255.0
10.0.0.1
end
```

INPUT :

INPUT (PC0 Command Prompt)

ping 192.168.2.2

OUTPUT :

Reply from 192.168.2.2 : bytes = 32
Reply from 192.168.2.2 : bytes = 32
Reply from 192.168.2.2 : bytes = 32
Reply from 192.168.2.2 : bytes = 32

RESULT :

Thus static routing was configured and tested successfully using Cisco Packet Tracer.

24/9/26

8. RIP ROUTING

Aim:

To simulate RIP (Routing Information Protocol) using Cisco Packet Tracer.

PROCEDURE:

1. Open Cisco Packet Tracer and create the network topology using two routers, two switches and two PCs
2. Connect the devices using suitable cables
3. Assign IP addresses to the PCs and the router interfaces.
4. Enable the router interfaces using no shutdown
5. Configure RIP version 2 on both routers.
6. Advertise the directly connected networks using the network command.
7. Test connectivity between the PCs using ping

RIP ROUTING:

RIP (Routing Information Protocol) is a legacy, interior gateway routing protocol used to automatically manage and discover paths within a small local network.

CONFIGURATION

Router 1

```
enable
configuration terminal
router rip
 version 2
 no auto-summary
 network 192.168.1.0
 network 10.0.0.0
 end
```

Router 2

```
enable
configure terminal
router rip
version 2
 no auto-summary
 network 192.168.2.0
 network 10.0.0.0
 end
```

INPUT:

INPUT (PC0 Command Prompt)

ping 192.168.2.2

OUTPUT:

Reply from 192.168.2.2: bytes = 32
Reply from 192.168.2.2: bytes = 32
Reply from 192.168.2.2 bytes = 32
Reply from 192.168.2.2 bytes = 32

RESULT:

Thus RIP routing was configured and tested Successfully using Cisco Packet Tracer.

Exp-No 12

25/9/26

END-TO-END COMMUNICATION AT TRANSPORT LAYER

Q) Echo client-server using TCP/UDP sockets.

Aim: To implement an echo client-server program using TCP/UDP sockets.

PROCEDURE:

1. Import the socket module
2. Create a server socket and bind it to an IP address and port.
3. Start the server and wait for client messages.
4. The client sends a message to the server.
5. The server sends the same message back to the client.
6. Display the received message.

TCP/UDP Sockets:

TCP and UDP sockets are the endpoints used by applications to send and receive data across a network. They operate at the Transport layer (layer 4) of the OSI model, acting as the bridge between your software code and the physical network infrastructure.

PROGRAM: - server.py (TCP)

```
import socket
s = socket.socket()
s.bind(("127.0.0.1", 5000))
s.listen(1)
c, addr = s.accept()
msg = c.recv(1024).decode()
print("Client", msg)
c.send(msg.encode())
c.close()
s.close()
```

PROGRAM - client.py (TCP)

```
import socket
s = socket.socket()
s.connect(("127.0.0.1", 5000))
msg = input("Enter message:")
s.send(msg.encode())
print("Echo:", s.recv(1024).decode())
s.close
```

input:

Enter message: Hello

output:
client: Hello
Echo : Hello

RESULT: Thus the echo client-server program was implemented using TCP sockets.

25/9/26

b) Chat Program using Socket Programming

Aim:

To implement a chat program using client-server Socket programming.

PROCEDURE:

1. Import the Socket module
2. Create a Server Socket and bind it to a port
3. Start the Server and accept a client connection
4. The client sends a message to the Server
5. The Server replies to the client.
6. Display the exchanged messages.. and close the connection.

SOCKET:

A Socket is an abstract software endpoint that allows two processes - either on the same computer or across the globe via the internet to communicate with each other.

CODE - server.py (TCP)

```
import socket
s = socket.socket()
s.bind(("127.0.0.1", 5001))
s.listen(1)
c, addr = s.accept()
while True:
    msg = c.recv(1024).decode()
    if msg.lower() == "bye":
        break
    print("client", msg)
    reply = input("Server : ")
    c.send(reply.encode())
    if reply.lower() == "bye":
        break
c.close() s.close()
```

CODE - client.py (TCP)

```
s = socket.socket()
s.connect(("127.0.0.1", 5001))
while True:
    msg = input("client: ")
    s.send(msg.encode())
    if msg.lower() == "bye":
        break
    reply = s.recv(1024).decode()
    print("Server", reply)
    if reply.lower() == "bye":
        break
s.close()
```

Input:

Client: Hello

Server: Hi

Client: How are you?

Server: Fine

Client: bye

output:

Client: Hello

Server: Hi

Client: How are you?

Server: Fine

RESULT:

Thus a Client - Server chat program was implemented Successfully using TCP sockets.

EXP. NO: 12

1/10/26

IMPLEMENT YOUR OWN PING PROGRAM

Aim:

To implement a ping program to check whether a host is reachable over a network.

PROCEDURE:

1. Import the os module
2. Get the destination host name or IP address from the user.
3. Execute the system ping command using the host address
4. Display the ping response
5. Verify whether the destination host is reachable

PING:

The ping program is a fundamental network diagnostic tool used to test if a host (a computer, server or router) is reachable across an IP network and to measure the round-trip time for messages sent to it.

PROGRAM (Python)

```
import os
host = input("Enter IP address or hostname: ")
os.system("ping -n 4" + host if os.name == "nt"
          else "ping -c 4" + host)
```

INPUT:

Enter IP address or hostname: 168.123.17.2

OUTPUT:

Pinging 168.123.17.3 with 32 bytes of data:
Reply from 168.123.17.3 bytes = 32 time = 2ms TTL=117
Reply from 168.123.17.3 bytes = 32 time = 19ms TTL = 117

Packets: sent = 4, Received = 4, lost = 0 (0% loss)

RESULT:

Thus the ping program was implemented successfully to check network connectivity to a destination host.

Exp.No: 14

8/10/26

PACKING SNIFFING USING RAW SOCKETS

Aim:

To write a program using RAW sockets to implement packet sniffing

PROCEDURE:

1. Import the socket module
2. Create a RAW socket
3. Receive packets from the network
4. Display the received packet data
5. Stop the program using ctrl+c

RAWSOCKET:

A raw socket is a special type of network socket that lets an application send and receive low-level IP packets directly, bypassing the operating system's standard transport-layer protocols like TCP or UDP

PROGRAM:

```
import socket
s = socket.socket(
    socket.AF_PACKET,
    socket.SOCK_RAW,
    socket.ntohs(3)
)
try:
    while True:
        packet, addr = s.recvfrom(65535)
        print("Packet received:", packet.hex())
except KeyboardInterrupt:
    s.close()
```

OUTPUT:

```
Packet received: 1616161
Packet received: 080045
Packet received: 001122
...
```

RESULT:

Thus packet sn
successfully using

OUTPUT:

Packet received : 16166666616...

Packet received: 0800450000 34...

Packet received: 00112233 4455...

..

RESULT:

Thus packet Sniffing was implemented successfully using RAW Sockets.

.NO:15

9/10/26

ANALYZE WEB LOGS USING WEBALIZER

Aim:
To analyse different types of web logs using the Webalizer tool.

WEBALIZER:
Webalizer is a popular, open-source web log analysis program that generates detailed, graphical usage statistics for websites.

PROCEDURE
1. Download and install the Webalizer tool
2. Open the Webalizer application.
3. Download a sample web server log file from the internet
4. Select the downloaded log file as the input
5. Configure the output directory if required
6. Click run webalizer to process the log file
7. View the generated report containing website access statistics
8. Analyze the number of hits, pages, visits, sites and bandwidth usage.

INPUT:

192.168.1.10 -- [10/Oct/2026:10:00:00 +0530] "GET /index.html HTTP/1.1" 200 1024

192.168.1.11 -- [10/Oct/2026:10:05:00 +0530] "GET /about.html HTTP/1.1" 200 2048

192.168.1.12 -- [10/Oct/2026:10:10:00 +0530] "GET /image.png HTTP/1.1" 200 4086

192.168.1.13 -- [10/Oct/2026:10:15:00 +0530] "GET /contact.html HTTP/1.1" 404 512

OUTPUT:

WEBALIZER STATISTICS REPORT

Total Hits : 4
Total Files : 3
Total Pages : 2
Total Visits : 3
Total Sites : 3
Total Bandwidth : 7680 bytes

- Requested Pages:

1. / index.html
2. / about.html
:

RESULT:

Thus Web server log files were analyzed successfully using the Webalizer tool.